EGYPTIAN CHINESE UNIVERSITY

Faculty of Engineering

Department of Distributed Systems

Distributed Computing Systems

Final Technical Documentation

DistroChat

A Distributed Multi-Client Chat System

Secure · Scalable · Fully Distributed

PROJECT TEAM

Abdallah Hegazy — 192200322
Ziad Ghanem — 192200345
Yossef Ayman — 192200339
Omar Rehabeldein — 192200340
Mohamed Zarref — 192200278

ACADEMIC SUPERVISION

Dr. Doaa Mabrouk
Eng. Mohamed Tarek
Eng. Mohamed Khatab

1. Executive Summary

> **Project Elevator Pitch**
> *DistroChat is more than just a chat app; it's a secure, scalable, and fully distributed communication platform that combines professional-grade encryption with a premium user experience and robust administrative tools.*

1.1 Project Overview

DistroChat is a fully distributed, multi-client real-time chat application engineered using Python's socket API and a custom TCP-based communication protocol. The system is architected around a multi-threaded server engine capable of concurrently managing dozens of authenticated users across multiple chat rooms while maintaining message integrity, security, and performance.

Built as a graduation project in distributed computing, DistroChat demonstrates the practical implementation of enterprise-grade distributed systems concepts — including fault-tolerant networking, encrypted data transmission, thread-safe persistence, and real-time administration — within a cohesive, polished software product.

1.2 Core Objectives

- Design and implement a fully functional distributed chat architecture using low-level socket programming
- Develop a custom, reliable JSON-based communication protocol with length-prefixed framing
- Integrate end-to-end AES encryption with SHA-256 integrity verification for all messages
- Build a responsive, modern graphical client using CustomTkinter with glassmorphism dark mode
- Implement a real-time administrative dashboard with live server analytics and moderation tools
- Provide a scalable data persistence layer using SQLite for users, messages, and room management

1.3 Key Features Summary

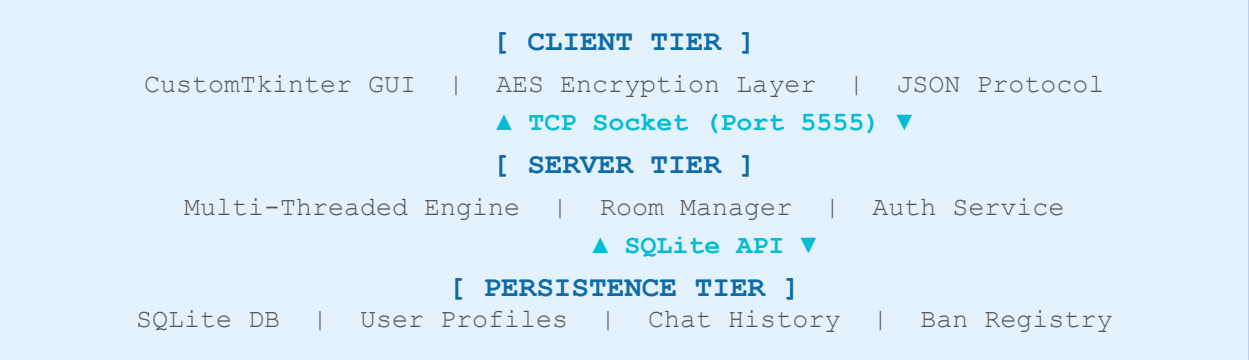
Feature	Technology	Status
Multi-client real-time messaging	Python Sockets + TCP	Complete
End-to-end encryption	AES-256 + SHA-256	Complete
User authentication	Bcrypt password hashing	Complete
Multi-room support	Thread-per-client engine	Complete
Direct messaging (DM)	Private channel routing	Complete
File & image sharing	Binary protocol extension	Complete
Admin dashboard	CustomTkinter + Matplotlib	Complete

Feature	Technology	Status
Persistent storage	SQLite3 + thread-safe ORM	Complete

2. Core Architecture

DistroChat employs a classic three-tier distributed architecture, separating concerns between the client presentation layer, the server processing engine, and the persistence layer. This design enables horizontal scalability and clear modularity for future system evolution.

2.1 System Architecture Diagram



2.2 Distributed Client-Server Model

The server operates as a single authoritative hub using a thread-per-client concurrency model. Each new TCP connection spawns a dedicated OS thread, allowing true parallel message processing across all connected clients. The server maintains shared state through thread-safe data structures and synchronized SQLite operations, preventing race conditions during concurrent access.

Server Engine Components

- Connection Listener — Binds to TCP port 5555, accepts incoming socket connections and hands off to worker threads
- Authentication Service — Validates login credentials against bcrypt hashes stored in SQLite
- Room Manager — Maintains active room membership maps and routes messages to correct subscriber sets
- Message Dispatcher — Thread-safe broadcast engine that fans out messages to all room members
- File Transfer Handler — Manages chunked binary transfers for images and documents
- Admin Controller — Exposes server metrics and moderation commands to the administration interface

2.3 Deployment Topology

Deployment Mode	Network Scope	Max Clients	Use Case
Local (LAN)	Single subnet	50+	Office / lab environment
WAN (Port-forwarded)	Internet-wide	200+	Remote team collaboration
Cloud VPS	Global	Scalable	Production deployment

2.4 Technology Stack

Layer	Component	Technology
Network	Socket Communication	Python socket module, TCP/IP
Concurrency	Multi-threading Engine	Python threading module
Security	Encryption & Hashing	PyCryptodome (AES), bcrypt, hashlib
Persistence	Database Layer	SQLite3 with WAL journal mode
GUI	Client Interface	CustomTkinter 5.x, tkinter
Analytics	Admin Graphs	Matplotlib, collections.deque

3. Custom Communication Protocol

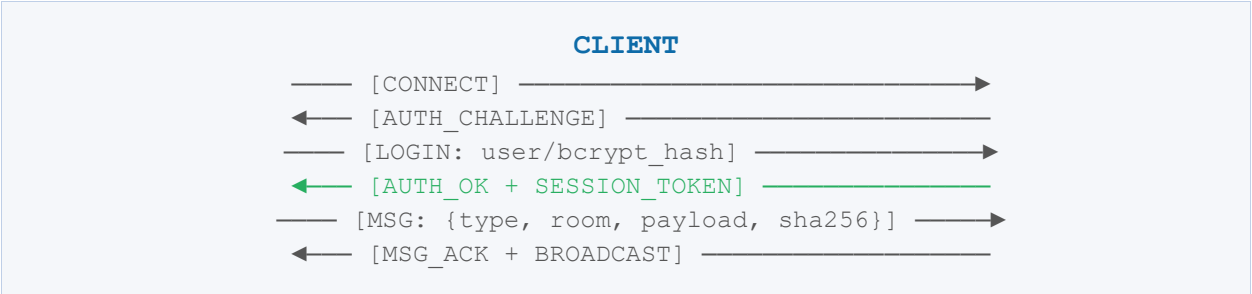
DistroChat defines a proprietary application-layer protocol built on top of TCP. The protocol uses JSON-encoded message bodies wrapped in a length-prefixed binary frame, ensuring reliable, complete, and ordered delivery of all communication primitives.

3.1 Packet Structure

Field	Size	Description
Length Prefix	4 bytes (big-endian)	Total length of the JSON payload in bytes
Message Type	String (JSON field)	Identifies packet purpose: MSG, AUTH, FILE, TYPING, STATUS
Sender ID	String (JSON field)	Authenticated username or session token
Room / Target	String (JSON field)	Destination room name or DM target username
Payload	String / Base64	Message content; binary files encoded as Base64
Checksum	String (JSON field)	SHA-256 hash of payload for integrity verification

Field	Size	Description
Timestamp	ISO-8601 (JSON field)	Server-assigned message timestamp

3.2 Communication Sequence Diagram



3.3 Message Types

- MSG — Standard chat message broadcast to a room or DM target, AES-encrypted payload
- FILE — Binary file transfer with chunked encoding; receiver reassembles from Base64 chunks
- TYPING — Lightweight real-time indicator (no persistence) signaling active user keystrokes
- STATUS — User online/offline/away presence updates broadcast to all room members
- AUTH — Authentication handshake: challenge-response with bcrypt verification
- ADMIN — Privileged server control commands (kick, ban, broadcast) from admin clients only
- ACK — Server acknowledgement confirming message receipt and successful delivery

3.4 Reliability Guarantees

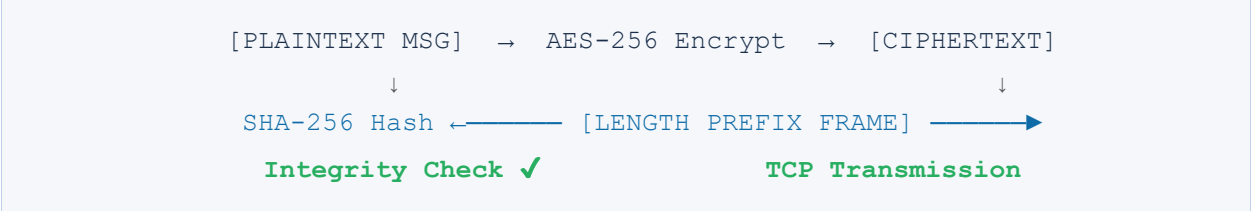
TCP's guaranteed ordered delivery forms the transport foundation. Above TCP, the length-prefix framing ensures complete packet boundaries are honored — no partial reads can corrupt message parsing. SHA-256 checksums provide an additional integrity layer, allowing the client to detect and discard any tampered or corrupted payload before displaying to the user.

4. Security & Privacy

Security Model

DistroChat implements a defense-in-depth security architecture combining transport-layer encryption, application-layer payload encryption, and cryptographic credential storage to protect user data at rest and in transit.

4.1 Encryption Architecture



4.2 AES End-to-End Encryption

All message payloads are encrypted using AES-256 in CBC mode via the PyCryptodome library. A shared symmetric key is negotiated during the authentication handshake and stored only in memory for the session duration. Messages are never stored or transmitted in plaintext — even in the SQLite database, message content is persisted in encrypted form.

Encryption Implementation Details

- **Algorithm: AES-256-CBC with PKCS7 padding**
- Key size: 256 bits (32 bytes) — derived using PBKDF2-HMAC-SHA256 with 100,000 iterations
- IV: Randomly generated 128-bit initialization vector per message
- Storage: Encrypted ciphertext + IV stored together; key never persisted to disk

4.3 Password Security (Bcrypt)

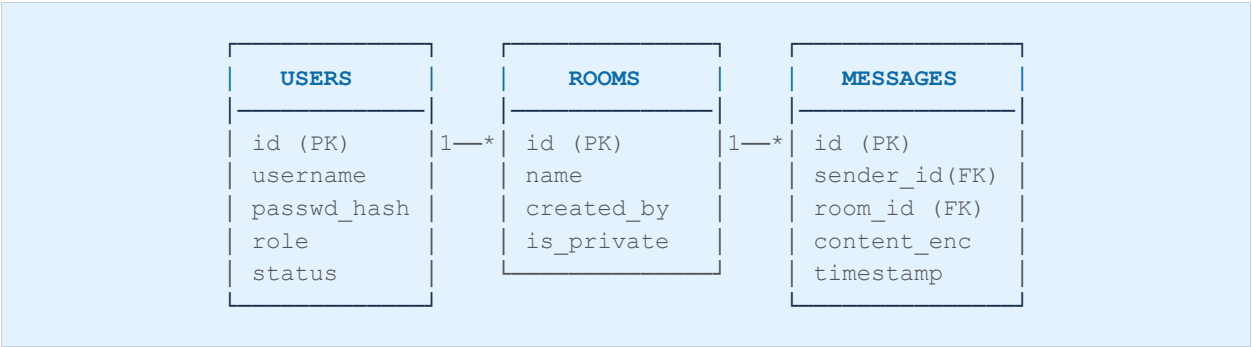
User passwords are never stored in plaintext or simple hashes. DistroChat uses the bcrypt adaptive hashing algorithm with a cost factor of 12, ensuring passwords remain secure against brute-force attacks even as hardware capabilities improve over time.

Security Mechanism	Algorithm	Purpose
Password Storage	Bcrypt (cost=12)	Resistance to brute-force and rainbow table attacks
Message Encryption	AES-256-CBC	End-to-end message confidentiality
Integrity Verification	SHA-256 HMAC	Tamper detection and payload validation
Session Tokens	UUID4 + HMAC-SHA256	Stateless authenticated session management

5. Data Persistence Layer

DistroChat uses SQLite as its embedded relational database engine, operating in WAL (Write-Ahead Logging) journal mode to support concurrent reads without blocking writes. All database operations are wrapped in thread-safe context managers with connection pooling to prevent data corruption under concurrent access.

5.1 Entity-Relationship Diagram



5.2 Database Schema

Table	Primary Key	Key Columns	Purpose
users	id (INT)	username, passwd_hash, role, avatar, status	User account registry with authentication data
rooms	id (INT)	name, created_by, is_private, created_at	Chat room definitions and ownership
room_members	Composite (user_id, room_id)	joined_at, role	Room membership and per-room roles
messages	id (INT)	sender_id, room_id, content_enc, iv, timestamp	AES-encrypted message archive
direct_messages	id (INT)	sender_id, receiver_id, content_enc, iv, read_at	Private one-to-one message storage
bans	id (INT)	user_id, banned_by, reason, expires_at	Ban registry with optional expiry
files	id (INT)	sender_id, room_id, filename, size_bytes, path	Shared file metadata and storage paths

5.3 Thread-Safe Database Operations

All database interactions are mediated through a singleton DatabaseManager class using Python's threading.Lock() for write serialization. Read operations use separate connection objects per thread (SQLite's check_same_thread=False with manual locking) to maximize read concurrency while preventing write conflicts.

6. User Interface & User Experience

The DistroChat client features a custom-designed graphical interface built with CustomTkinter 5.x, implementing a glassmorphism-inspired dark mode aesthetic with a professional blue-and-white color palette. The interface prioritizes usability through a clean three-panel layout: sidebar navigation, main chat window, and contextual right panel.

6.1 UI Architecture

UI Component	Framework Element	Description
Login Screen	CTkFrame + animations	Animated entrance with ECU branding and bcrypt auth
Main Chat Window	CTkScrollableFrame	Scrollable bubble-style message display with timestamps
Room Sidebar	CTkListbox	Dynamic room list with unread badge counters
Input Bar	CTkEntry + CTkButton	Rich text input with emoji picker and file attachment
User List Panel	CTkFrame	Live online/offline presence indicators per room
Typing Indicator	CTkLabel (animated)	Animated three-dot indicator for active typers
Admin Dashboard	Toplevel + Matplotlib	Separate admin window with live graphs and controls

6.2 Design System

- Color Scheme: Dark navy primary (#1A2E4A) with electric blue accents (#2196F3) and teal highlights
- Typography: Inter font family for body text, monospace for usernames and timestamps
- Glassmorphism: Translucent panel overlays with blur effects on modal windows
- Responsive layout adapts to window resize events with minimum 800x600 constraint
- Accessibility: High-contrast mode toggle, keyboard navigation support throughout

6.3 User Workflow

The client application follows a linear onboarding flow: launch screen → server address entry → login / registration → main chat interface. Once authenticated, users land in the General room by default and can navigate to other rooms, open direct messages, share files, and view user profiles without leaving the application window.

7. Administration Dashboard

The DistroChat administration interface provides privileged operators with real-time visibility into server health, active user sessions, message throughput, and moderation controls. The dashboard is accessible only to users with the ADMIN role and opens as a separate application window.

7.1 Dashboard Modules

Module	Update Interval	Data Source	Description
Live User Counter	1 second	Server memory	Real-time count of authenticated sessions
Message Rate Graph	5 seconds	Server metrics	Rolling 60-second messages-per-second chart
Room Activity	10 seconds	SQLite aggregates	Bar chart of messages per room
CPU & Memory	2 seconds	psutil library	Server process resource consumption
User Session Table	Real-time	Thread registry	IP, username, connection time, room membership
Ban Management	On-demand	SQLite bans table	Active bans with revoke/extend controls

7.2 Moderation Controls

- Kick User — Immediately terminates a user's socket connection; session tokens are invalidated
- Temporary Ban — Applies a time-limited ban (minutes to days) with reason logging in the bans table
- Permanent Ban — Blacklists a username and IP address combination from future connections
- Server Broadcast — Sends a system-wide announcement message to all connected users simultaneously
- Room Lockdown — Prevents non-admin users from posting to a specified room (read-only mode)
- Force Disconnect All — Emergency server-wide disconnection for security incident response

8. Scalability & Future Enhancements

8.1 Current Scalability Characteristics

The current thread-per-client model scales efficiently to approximately 200 concurrent users on modern server hardware (4 cores, 8GB RAM). Python's GIL limits CPU-bound parallelism, but since DistroChat's server workload is primarily I/O-bound (network reads and SQLite writes), real-world concurrency is not significantly constrained by the GIL.

8.2 Planned Architectural Enhancements

- **Async I/O Migration — Refactor server to Python asyncio for event-driven I/O, eliminating thread overhead and supporting 1000+ concurrent connections**
- **Horizontal Scaling — Deploy multiple server nodes behind a load balancer with shared Redis pub/sub for cross-node message routing**
- **Cloud Deployment — Containerize with Docker and deploy on AWS/GCP with auto-scaling groups and managed PostgreSQL replacing SQLite**
- **End-to-End Ratchet Encryption — Upgrade to Signal Protocol's Double Ratchet algorithm for forward secrecy in all DM conversations**
- **AI Moderation — Integrate transformer-based toxicity detection model for automatic flagging of policy-violating content**
- **Voice & Video — Add WebRTC data channels for peer-to-peer audio/video within the chat interface**
- **REST API Gateway — Expose server functionality via a RESTful API for third-party client integration and webhooks**

8.3 Migration Roadmap

Phase	Timeline	Deliverable	Impact
Phase 1	Q3 2025	Asyncio server refactor	Support 1000+ concurrent users
Phase 2	Q4 2025	Docker containerization	Cloud-native deployment
Phase 3	Q1 2026	Horizontal load balancing	Near-infinite horizontal scale
Phase 4	Q2 2026	AI moderation engine	Automated content safety

9. Conclusion

DistroChat successfully demonstrates the practical application of distributed computing principles within a cohesive, functional, and visually polished software product. From low-level socket programming and custom protocol design to AES encryption, thread-safe database persistence, and real-time graphical interfaces — every component reflects careful engineering consideration and academic rigor.

The project achieves its primary technical goals: a working distributed chat system capable of supporting dozens of simultaneous users, full end-to-end message encryption, a professional-grade user interface, and a comprehensive administrative toolkit. The codebase is modular, well-documented, and structured for expansion.

From an academic perspective, DistroChat bridges theoretical distributed systems concepts — consistency, concurrency, fault tolerance, and security — with hands-on implementation. The challenges encountered and solved during development (deadlock avoidance in multi-threaded SQLite access, reliable packet framing over TCP, and AES key exchange) represent genuine distributed systems engineering problems with real-world applicability.

> **Academic Significance**
> *DistroChat represents a complete, deployment-ready distributed communication platform engineered from first principles — demonstrating mastery of networking, security, concurrency, and software design at the graduation project level.*

References

[1] Tanenbaum, A. S., & Van Steen, M. (2017). Distributed Systems: Principles and Paradigms (3rd ed.). Pearson.

[2] Beazley, D., & Jones, B. K. (2013). Python Cookbook. O'Reilly Media.

[3] Preneel, B. (2011). Cryptographic Hash Functions. Springer Encyclopedia of Cryptography and Security.

[4] PyCryptodome Documentation. (2024). AES Encryption in Python. https://pycryptodome.readthedocs.io

[5] Python Software Foundation. (2024). socket — Low-level networking interface. Python 3.12 Documentation.

[6] SQLite Consortium. (2024). SQLite Documentation: WAL Mode. https://sqlite.org/wal.html

[7] CustomTkinter. (2024). Modern Tkinter Framework Documentation. https://customtkinter.tomschimansky.com

[8] Provos, N., & Mazières, D. (1999). A Future-Adaptable Password Scheme. USENIX Annual Technical Conference.

[9] Stevens, W. R., Fenner, B., & Rudoff, A. M. (2003). UNIX Network Programming, Volume 1. Addison-Wesley.

[10] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.